

# 정보 수업 정리 — 자료구조와 재귀

큐 / 스택 · 재귀함수 · 콜스택 · 예제 코드 (gcd, lcm, stars)

## 목차

1. 큐(Queue)와 스택(Stack)
2. 큐·스택의 변종들
3. 재귀함수의 구성요소
4. 선행 vs 후행: 동작 위치에 따른 차이
5. 콜스택(Call Stack)
6. 예제 1: 최대공약수(gcd)
7. 예제 2: 최소공배수(lcm)
8. 예제 3: 별 찍기(stars) — 선행/후행 비교
9. 예제 3 변환: for문으로 바꾸기

## 1. 큐(Queue)와 스택(Stack)

둘 다 데이터를 한 줄로 쌓아두고 꺼내 쓰는 자료구조지만, **꺼내는 순서**가 다릅니다.

| 구분  | 큐 (Queue)                  | 스택 (Stack)                |
|-----|----------------------------|---------------------------|
| 원칙  | FIFO (First In, First Out) | LIFO (Last In, First Out) |
| 비유  | 줄 서기 (먼저 온 사람이 먼저 나감)      | 접시 쌓기 (마지막에 올린 접시부터 사용)   |
| 넣기  | enqueue (뒤에 추가)            | push (위에 추가)              |
| 꺼내기 | dequeue (앞에서 제거)           | pop (위에서 제거)              |

예시: [1, 2, 3] 순서로 넣었을 때

- 큐에서 꺼내면 → 1, 2, 3
- 스택에서 꺼내면 → 3, 2, 1

## 2. 큐·스택의 변종들

## 덱 (Deque, Double-Ended Queue)

양쪽 끝에서 모두 넣고 꺼낼 수 있는 자료구조. "양방향 큐"라고도 부릅니다.

- 앞쪽: `push_front`, `pop_front`
- 뒤쪽: `push_back`, `pop_back`
- 큐로도, 스택으로도 쓸 수 있는 만능 구조. Python에서는 `collections.deque` 로 제공.

## 우선순위 큐 (Priority Queue)

들어온 순서가 아니라 **우선순위가 높은 것부터** 꺼냅니다.

- 예: 응급실 — 늦게 와도 위중한 환자가 먼저 진료받음.
- 보통 **힙(Heap)** 자료구조로 구현. Python에서는 `heapq` 모듈.

## 원형 큐 (Circular Queue)

고정 크기 배열을 끝과 시작이 이어진 원형으로 사용하는 큐. 메모리를 효율적으로 재활용할 수 있습니다.

**정리** 큐·스택은 "어떤 순서로 꺼낼 것인가"의 차이. 거기에 "양쪽에서 꺼낼 수 있게"(덱), "중요한 것부터 꺼낼 수 있게"(우선순위 큐) 같은 규칙을 더한 것이 변종들입니다.

## 3. 재귀함수의 구성요소

재귀함수(recursive function)는 **자기 자신을 호출하는 함수**입니다. 잘 동작하려면 세 가지가 모두 필요합니다.

1. **종료 조건 (base case)** — 더 이상 재귀하지 않고 멈추는 조건. 없으면 무한히 자기 자신을 부르다 콜스택이 터집니다.
2. **재귀 호출 (recursive call)** — 문제를 더 작은 문제로 쪼개서 자기 자신을 호출하는 부분.
3. **동작 (work)** — 출력, 계산, 누적 등 함수가 실제로 수행하는 일.

```
def recur(n):  
    if n <= 0:          # ❶ 종료 조건  
        return  
    print(n)            # ❸ 동작 (선행)  
    recur(n - 1)        # ❷ 재귀 호출  
    # print(n)          # ❸ 동작을 여기 두면 후행
```

## 4. 선행 vs 후행: 동작 위치에 따른 차이

"동작"이 재귀 호출 **전에** 있는지 **후에** 있는지에 따라 결과가 완전히 달라집니다. 수업에서 **선행 / 후행**이라고 불렀는데, 이 용어에 대한 표준 정리:

### 표준 용어가 있는가?

한국어로 통일된 표준 용어는 **없습니다**. 교과서마다 표현이 다릅니다. 대신 영어권에는 트리 순회 (tree traversal)에서 유래한 두 가지 표현이 널리 쓰입니다.

| 한국어 표현 (수업/교재마다 다름) | 영어 표현 (널리 쓰임)                   |
|---------------------|---------------------------------|
| 선행 / 후행             | —                               |
| 전위 / 후위             | pre-order / post-order          |
| 하향식 / 상향식           | top-down / bottom-up            |
| 들어가며 처리 / 돌아오며 처리   | on the way down / on the way up |

**참고** 알고리즘 책에서는 보통 **pre-order (전위) / post-order (후위)**라는 표현을 가장 많이 씁니다. 트리 순회에서 빌려온 용어인데, 재귀의 동작 위치를 가리킬 때도 자연스럽게 씁니다.

### 선행 (pre-order)

```
def f(n):  
    if n == 0: return  
    print(n)    # 동작  
    f(n - 1)    # 재귀
```

f(3) → 출력: 3 2 1

들어가는 길에 일을 처리. 호출 순서대로 결과가 나옴.

### 후행 (post-order)

```
def f(n):  
    if n == 0: return  
    f(n - 1)    # 재귀  
    print(n)    # 동작
```

f(3) → 출력: 1 2 3

나오는 길에 일을 처리. 가장 깊이 들어간 곳부터 거꾸로 결과가 나옴.

## 5. 콜스택 (Call Stack)

함수가 호출될 때마다 컴퓨터는 그 함수의 정보(매개변수, 지역변수, 돌아갈 위치 등)를 **스택**에 쌓습니다. 이 스택을 **콜스택**이라고 합니다.

### 호출과 실행의 순서

- 함수가 호출되면 → 콜스택 위에 **쌓임 (push)**

- 함수가 끝나면 → 콜스택 맨 위에서 제거됨 (pop)
- 항상 맨 위에 있는 함수가 실행 중

예시: `f(3)` 이 호출될 때 콜스택의 변화

시간 →

|                                                                                                  |                                            |
|--------------------------------------------------------------------------------------------------|--------------------------------------------|
| [ <code>f(3)</code> ]                                                                            | 호출 시작                                      |
| [ <code>f(2)</code> ]<br>[ <code>f(3)</code> ]                                                   | <code>f(3)</code> 안에서 <code>f(2)</code> 호출 |
| [ <code>f(1)</code> ]<br>[ <code>f(2)</code> ]<br>[ <code>f(3)</code> ]                          | <code>f(2)</code> 안에서 <code>f(1)</code> 호출 |
| [ <code>f(0)</code> ]<br>[ <code>f(1)</code> ]<br>[ <code>f(2)</code> ]<br>[ <code>f(3)</code> ] | <code>f(1)</code> 안에서 <code>f(0)</code> 호출 |
| [ <code>f(1)</code> ]<br>[ <code>f(2)</code> ]<br>[ <code>f(3)</code> ]                          | <code>f(0)</code> 종료, 제거됨                  |
| [ <code>f(2)</code> ]<br>[ <code>f(3)</code> ]                                                   | <code>f(1)</code> 종료                       |
| [ <code>f(3)</code> ]                                                                            | <code>f(2)</code> 종료                       |
| (비어있음)                                                                                           | <code>f(3)</code> 종료                       |

**핵심** 호출은 위에서부터 쌓이고, 종료는 위에서부터 빠진다. 그래서 가장 마지막에 호출된 함수가 가장 먼저 끝납니다(LIFO). 후행(post-order) 동작이 "거꾸로" 출력되는 이유가 바로 이 콜스택의 LIFO 성질 때문입니다.

## 6. 예제 1: 최대공약수 (gcd)

유클리드 호제법: 두 수 `a`, `b` 의 최대공약수는 `b` 와 `a%b` 의 최대공약수와 같다. `b` 가 0이 되면 `a` 가 답.

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)
```

## 동작 추적: gcd(24, 18)

| 호출           | 계산               | 반환값 |
|--------------|------------------|-----|
| gcd(24, 18)  | 18 != 0          | ↑ 6 |
| → gcd(18, 6) | 6 != 0           | ↑ 6 |
| → gcd(6, 0)  | b == 0, return 6 | ↑ 6 |
| 최종 결과: 6     |                  |     |

호출이 점점 깊어지다가 `b == 0` 이 되는 순간(종료 조건) 값이 정해지고, 그 값이 콜스택을 거슬러 올라가며 그대로 반환됩니다.

## 7. 예제 2: 최소공배수 (lcm)

최소공배수의 정의를 그대로 풀어서 재귀로 만들어봅시다. 두 수 `a, b` 중 작은 쪽에 자신의 원래 값을 계속 더해서 두 값이 같아질 때까지 반복합니다.

### 아이디어

- `a == b` 이면 그 값이 바로 공배수 → 종료.
- `a > b` 이면 **작은 쪽 b**에 b의 원래 값을 더해서 키운다.
- `a < b` 이면 **작은 쪽 a**에 a의 원래 값을 더해서 키운다.

여기서 중요한 점: 재귀 호출이 진행되면서 `a` 와 `b` 값은 계속 바뀝니다. 그런데 더할 때는 **처음 입력값**을 더해야 하므로, 원래 값을 어딘가에 따로 저장해둬야 합니다. 이때 **전역변수(global)**를 사용합니다.

```
def lcm(a, b):  
    global n, m      # 처음 입력값을 보존  
    if a == b:  
        return a  
    if a > b:  
        return lcm(a, b + m)  
    else:  
        return lcm(a + n, b)  
  
n, m = 4, 6  
print(lcm(n, m))
```

**왜 global인가?** 매개변수 `a, b` 는 호출이 거듭될수록 값이 바뀝니다( `4 → 8 → 12 ...` ). 하지만 "더할 양"은 항상 **처음 값**이어야 하므로, 함수 안에서 매개변수만 봐서는 알 수 없습니다. 그래서 처음 값을 `n, m` 이라는 전역변수에 저장해두고, 재귀 안에서 그것을 참조합니다.

동작 추적: `lcm(4, 6)` ( $n=4, m=6$ )

| 호출                       | 비교                       | 다음 호출                                   |
|--------------------------|--------------------------|-----------------------------------------|
| <code>lcm(4, 6)</code>   | <code>4 &lt; 6 →</code>  | <code>lcm(4+4, 6) = lcm(8, 6)</code>    |
| <code>lcm(8, 6)</code>   | <code>8 &gt; 6 →</code>  | <code>lcm(8, 6+6) = lcm(8, 12)</code>   |
| <code>lcm(8, 12)</code>  | <code>8 &lt; 12 →</code> | <code>lcm(8+4, 12) = lcm(12, 12)</code> |
| <code>lcm(12, 12)</code> | <code>12 == 12 →</code>  | <code>return 12</code>                  |

최종 결과: 12

## 이 코드는 어떤 스타일인가?

- 종료 조건이 **같아지는 순간**이라는 점이 직관적입니다(공배수의 정의 그대로).
- 전역변수를 쓰는 방식은 코드가 짧아지지만, 같은 함수를 여러 번 다른 값으로 호출하기 어렵다는 단점이 있습니다(매번 `n, m` 을 다시 세팅해야 함).
- 실무에서는 보통 `lcm(a, b) = a * b // gcd(a, b)` 공식을 쓰지만, **재귀의 동작 원리를 이해하는 학습 목적**으로는 이 방식이 훨씬 좋은 연습이 됩니다.

## 8. 예제 3: 별 찍기 (stars) — 선행/후행 비교

같은 구조의 함수에서 `print` 위치만 바뀌도 결과가 완전히 달라집니다.

### 선행 (print가 재귀 앞에)

```
def stars(n):  
    if n == 0:  
        return  
    print('*' * n)  
    stars(n - 1)
```

stars(4) 결과:

```
****  
***  
**  
*
```

### 후행 (print가 재귀 뒤에)

```
def stars(n):  
    if n == 0:  
        return  
    stars(n - 1)  
    print('*' * n)
```

stars(4) 결과:

```
*  
**  
***  
****
```

## 왜 이렇게 되는가?

선행에서는 `stars(4)` 가 호출되자마자 `print('****')` 가 먼저 실행되고, 그 다음 `stars(3)` 이 호출됩니다. 즉 **들어가는 길에** 출력이 일어나서 큰 수부터 찍힙니다.

후행에서는 `stars(4)` 가 `stars(3)` 을 먼저 부르고, 그게 또 `stars(2)` 를 부르고... 하면서 가장 깊이 (`stars(0)` )까지 내려갑니다. 그 후 콜스택을 거슬러 올라오면서(LIFO) `stars(1)` 의 `print` → `stars(2)` 의 `print` → ... 순으로 실행되어 작은 수부터 찍힙니다.

[ 후행 동작의 콜스택 흐름 - `stars(4)` ]

| 들어갈 때:                         | 나올 때 (print 실행):               |
|--------------------------------|--------------------------------|
| <code>stars(4)</code> 호출 →     | ← <code>print('****')</code> ④ |
| <code>stars(3)</code> 호출 →     | ← <code>print('***')</code> ③  |
| <code>stars(2)</code> 호출 →     | ← <code>print '**')</code> ②   |
| <code>stars(1)</code> 호출 →     | ← <code>print('*')</code> ①    |
| <code>stars(0)</code> → return |                                |

## 9. 예제 3 변환: for문으로 바꾸기

재귀로 짠 코드는 보통 반복문으로 바꿀 수 있습니다. `stars` 도 마찬가지입니다.

### 선행 버전 → for문

출력이 `n, n-1, ..., 1` 순서이므로 `range` 를 거꾸로 돌립니다.

```
def stars(n):
    for i in range(n, 0, -1):
        print('*' * i)
```

### 후행 버전 → for문

출력이 `1, 2, ..., n` 순서이므로 `range` 를 정방향으로 돌립니다.

```
def stars(n):
    for i in range(1, n + 1):
        print('*' * i)
```

**생각해볼 점** 재귀의 선행은 보통 정방향 반복문으로 자연스럽게 바뀝니다(들어가는 길의 순서 그대로). 반면 후행은 콜스택의 LIFO 때문에 결과가 "거꾸로" 나오므로, for문으로 바꿀 때 방향을 뒤집거나 결과

를 모아서 거꾸로 출력해야 합니다. `stars` 처럼 단순한 경우는 `range` 방향만 바꾸면 되지만, 복잡한 후행 재귀(예: 트리 후위 순회)는 명시적인 스택 자료구조가 필요합니다.